Profesor de prácticas: Francisco Jose Montoya Dato

Miguel Ángel Tornel Martínez
Grupo 1, Subgrupo 2
mangel.tornel@um.es
DNI: 49971455E

Samuel Espín Santos
Grupo 1, Subgrupo 2
samuel.espins@um.es
DNI: 49476422H

Índice

1. Enunciado del problema **2**

1.1. Problema asignado 2

1.2. Descripción del problema 2

2. Diseño **2**

2.1. Diseño Iterativo 2

2.2. Diseño Divide y Vencerás 3

3. Implementación **4**

4. Validación **5**

4.1. Casos de prueba unitarios 5

4.2. Oráculo 6

5. Análisis Teórico **7**

5.1. Definición del tamaño del problema 7

5.2. Tamaños de los subproblemas 7

5.3. Coste del algoritmo 8

5.4. Conclusión 8

6. Análisis experimental **10**

7. Contraste **11**

1. Enunciado del problema

1.1. Problema asignado

Número de ejercicio: $49971455 + 49476422 = 99447877 \text{ mód } 10 + 1 = 8$

1.2. Descripción del problema

Dada una cadena A con n caracteres y un conjunto S de 5 caracteres distintos, devolver todas las subcadenas de A formadas por los 5 elementos de S sin repetir (permutaciones sin repetición). Habrá que obtener el número de subcadenas y su posición en la cadena A.

- Entrada: A (Cadena) y S (Conjunto)
- Salida: Subcadenas válidas y en que posiciones
- Ejemplo:
  - Entrada:
    - $A =$ `acbfabcedfcbcbadec`
    - $S = \{a, b, c, d, e\}$
  - Salida: 3 subcadenas válidas en las posiciones **5,13,14** (Empezando desde 1)

2. Diseño

En este apartado veremos el diseño de los dos algoritmos: Un algoritmo iterativo y un algoritmo basado en el divide y vencerás que se apoyará de la versión iterativa para casos atómicos que no se puedan dividir más y se deba resolver directamente mediante iterativo.

2.1. Diseño Iterativo

En este algoritmo se muestra cómo la versión iterativa resuelve el problema de contar las subcadenas y obtener las posiciones en las que empiezan. La idea es recorrer la cadena carácter a carácter. Para cada carácter comprobamos si pertenece al conjunto y lo guardamos en un historial de los últimos 5 caracteres. Esto nos permite comprobar si en ese grupo de caracteres se cumplen las condiciones del problema. El historial se mantiene utilizando una cola (FIFO). Cuando se añade un nuevo carácter y ya hay más de 5, se elimina el más antiguo. Mientras recorremos la cadena también llevamos un contador de cuántos caracteres distintos del conjunto aparecen en nuestro historial de 5 últimos caracteres. Si en algún momento ese número es igual a 5, significa que forman una subcadena válida, por lo que aumentamos el número de soluciones y guardamos su posición de inicio.

Esta versión iterativa también se utilizará para resolver algunos casos concretos dentro del algoritmo de Divide y Vencerás.

Algorithm 1 Número de subcadenas válidas y posiciones (iterativo)

```
function ITERATIVO(S, A, inicio, final)
    n ← final − inicio + 1
    letraContador ← diccionarioVacio
    historial ← colaVacia
    coincidencias ← 0
    numSubcadenas ← 0
    posiciones ← arrayVacio
    for i ← inicio to final do
        letra ← cadena[i]
        historial ← push(letra)
        if S ← contains(letra) then
            letraContador ← letraContador[letra] + 1
            if letraContador[letra] == 1 then
                coincidencias ← coincidencias + 1
            end if
        end if
        if historial ← size() > TAM_CONJUNTO then
            letraSale ← historial ← front()
            historial ← pop()
            if S ← contains(letraSale) then
                letraContador ← letraContador[letra] − 1
                if letraContador[letra] == 0 then
                    coincidencias ← coincidencias − 1
                end if
            end if
        end if
        if coincidencias == TAM_CONJUNTO then
            numSubcadenas ← numSubcadenas + 1
            posciones ← push(i − TAM_CONJUNTO + 2)
        end if
    end for
    return {numSubcadenas, posiciones}
end function
```

2.2. Diseño Divide y Vencerás

En este algoritmo se muestra una versión resumida de cómo se resolvería el problema usando la técnica de Divide y Vencerás. Primero se comprueban los casos base. Si la longitud de la cadena es menor que el tamaño del conjunto (5), entonces no puede haber ninguna solución. El otro caso base es cuando la longitud está entre 5 y 8. En este caso el problema se resuelve directamente con el algoritmo iterativo, ya que si dividiéramos una cadena tan pequeña en dos mitades obtendríamos subcadenas demasiado cortas para

contener soluciones. En el resto de casos el problema se divide en dos subproblemas: la mitad izquierda y la mitad derecha de la cadena. Se llama recursivamente al mismo procedimiento para resolver cada mitad. Finalmente se combinan las soluciones obtenidas en la parte izquierda, derecha para obtener el resultado final.

Algorithm 2 Número de subcadenas válidas y posiciones (Divide y Vencerás)

```
function DYV(S, A, inicio, final)
    n ← final − inicio + 1
    //Caso base 1
    if n < 5 then return {0, posicionesVacias}
    end if
    //Caso base 2
    if n ≤ 8 then return ITERATIVO(S, A, inicio, final)
    end if
    indexMitad ← inicio + (n/2)
    resIzq ← DYV(S, A, inicio, indexMitad − 1)
    resDer ← DYV(S, A, indexMitad, final)
    return MERGE(resIzq, resDer, S, A, indexMitad)
end function
```

Después de resolver ambas partes, puede ocurrir que aparezcan nuevas soluciones en la frontera entre las dos mitades, es decir, subcadenas que empiezan en la izquierda y terminan en la derecha. Para comprobar estas posibles soluciones se analiza una pequeña zona alrededor de la división. Esta zona tiene como máximo longitud 8, por lo que se resuelve utilizando el algoritmo iterativo. La función merge de sumar todas las subcadenas válidas que encontró izquierda, medio y derecha. Respecto a las posiciones de inicio de cada subcadena reutilizamos izquierda para insertar primero los de medio y luego los de derecha de esta manera tenemos la certeza que las posiciones iniciales de cada subcadena válida estan ordenadas.

Algorithm 3 Función auxiliar para combinar resultados en Divide y Vencerás

```
function MERGE(izq, der, cent)
    //Frontera
    centro ← ITERATIVO(8)
    //Combinar
    izq.numSubcadenas ← izq.numSubcadenas + der.numSubcadenas
    izq.numSubcadenas ← izq.numSubcadenas + cent.numSubcadenas
    izq.posiciones ← insert(izq.posiciones.end(), der.posiciones.begin(), der.posiciones.end())
    izq.posiciones ← insert(izq.posiciones.end(), cent.posiciones.begin(), cent.posiciones.end())
end function
```

3. Implementación

El proyecto asociado a esta memoria tiene los siguientes archivos[1] :

[1] Todo el código se encuentra en `https://github.com/MangelDeveloper/AED-2---DyV`

- Ficheros **Algoritmo.h** y **Algoritmo.cpp**: Estos ficheros contienen las cabeceras y las implementaciones de los algoritmos propuestos.
- Fichero **inputTest.txt**: Contiene las entradas utilizadas en cada test unitario.
- Fichero **outputTest.txt**: Almacena las salidas generadas por cada test unitario.
- Ficheros **Controlador.cpp** y **Controlador.h**: Estos ficheros se encargan de generar datos aleatorios, como cadenas y conjuntos de caracteres, así como de realizar test oráculo y medir tiempos de ejecución.
- Fichero **EstudioAlgoritmo.R**: El código de este fichero se encarga de generar gráficas a partir de los tiempos y de hacer los ajustes de regresión.
- Fichero **tiempoMejor.txt**: Contiene los tiempos de ejecución del algoritmo en el mejor caso, para que el fichero **EstudioAlgoritmo.R** pueda generar la gráfica correspondiente.
- Fichero **tiempoPeor.txt**: Contiene los tiempos de ejecución del algoritmo en el peor caso, para que el fichero **EstudioAlgoritmo.R** pueda generar la gráfica correspondiente.
- Fichero **Makefile**: Este fichero se encarga de como debe compilar el proyecto.

4. Validación

Para validar el algoritmo de DyV se han utilizado dos estrategias: Validación con casos de prueba unitarios y validación usando la función iterativa como oráculo.

4.1. Casos de prueba unitarios

Ningún caracter pertenece a S (Mejor caso) :

- Entrada: "zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz"
- Conjunto: $\{a, b, c, d, e\}$
- Salida esperada (Iterativo): 0 Subcadenas válidas
- Resultado (DyV): 0 Subcadenas válidas

Todos los caracteres pertenecen a S y todas las posiciones forman subcadenas válidas menos las 4 posiciones últimas (Peor caso) :

- Entrada: "**abcdeabcdeabcdeabcdeabcdeab**cdea"

- Conjunto: $\{a, b, c, d, e\}$
- Salida esperada (Iterativo): 27 Subcadenas válidas , Posiciones: 1 2 3 $\cdots$ 27
- Resultado (DyV): 27 Subcadenas válidas , Posiciones: 1 2 3 $\cdots$ 27

Otro conjunto con 1 subcadena válida:

- Entrada: "lkmna**cbfad**vxsa"
- Conjunto: $\{c, b, f, a, d\}$
- Salida esperada (Iterativo): 1 Subcadena válida , Posición: 6
- Resultado (DyV): 1 Subcadena válida , Posición: 6

Cadenas menores de tamaño 5:

- Entrada: "men"
- Conjunto: $\{a, b, c, d, e\}$
- Salida esperada (Iterativo): 0 Subcadenas válidas
- Resultado (DyV): 0 Subcadenas válidas

4.2. Oráculo

Para comprobar la corrección del algoritmo de Divide y Vencerás, se utiliza la función iterativa como oráculo. Esto significa que, para cada entrada aleatoria, comparamos los resultados obtenidos por DyV con los resultados que arroja el algoritmo iterativo, que se considera correcto por construcción.

Más concretamente, hemos rellenado valores aleatorios de caracteres desde la 'a' hasta la 'z' en una cadena de longitud 5, 10, 20, 50, 100 y 1000. Para cada una de estas 6 cadenas:

- Se ejecuta `SUBSECUENCIA_MAS_LARGA_ITERATIVO` con los mismos parámetros de entrada que `SUBSECUENCIA_MAS_LARGA_DYV`.
- Se comparan los valores devueltos: número de subcadenas válidas y posiciones en la cadena.
- Si ambos resultados coinciden en todas las pruebas, se confirma que el algoritmo DyV es correcto para esas entradas.

5. Análisis Teórico

5.1. Definición del tamaño del problema

Sea el tamaño del problema:

$$n = final - inicio + 1$$

Si dividimos la cadena en dos subproblemas obtenemos que:

$$indexMitad = inicio + \left\lfloor \frac{n}{2} \right\rfloor$$

Además hay que tener en cuenta las posibles subcadenas que se pueden formar entre los 8 caracteres del centro.

Entonces los subproblemas son:

- **Izquierdo:** $[inicio, indexMitad - 1]$
- **Derecho:** $[indexMitad, final]$

5.2. Tamaños de los subproblemas

Para el primer subproblema tenemos que:

$$n_1 = (indexMitad - 1) - inicio + 1 = indexMitad - inicio$$

Si sustituimos $indexMitad$ por su correspondiente valor obtenemos que:

$$n_1 = indexMitad - inicio = inicio + \left\lfloor \frac{n}{2} \right\rfloor - inicio = \left\lfloor \frac{n}{2} \right\rfloor$$

Para el segundo subproblema tenemos que:

$$n_2 = final - indexMitad + 1$$

Si sustituimos $indexMitad$ por su correspondiente valor obtenemos que:

$$n_2 = final - indexMitad + 1 = final - \left(inicio + \left\lfloor \frac{n}{2} \right\rfloor\right) + 1$$

$$final - \left(inicio + \left\lfloor \frac{n}{2} \right\rfloor\right) + 1 = final - inicio + 1 - \left\lfloor \frac{n}{2} \right\rfloor$$

$$final - inicio + 1 - \left\lfloor \frac{n}{2} \right\rfloor = n - \left\lfloor \frac{n}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil$$

Obteniendo así que los tamaños de los subproblemas son $n_1 = \left\lfloor \frac{n}{2} \right\rfloor$ y $n_2 = \left\lceil \frac{n}{2} \right\rceil$

5.3. Coste del algoritmo

Definiendo el algoritmo de combinación como $m(n-4)$ su coste es:

$$m\,(n-4) \in \ \Theta\,(n)$$

Dentro de $m(n-4)$ se calcula la frontera, donde definimos que será del rango de

$$[indexMitad - 4, indexMitad + 3]$$

al ser rango de 8 elementos, debemos resolverlo directamente mediante la versión iterativa:

$(indexMitad + 3) - (indexMitad - 4) + 1 = indexMitad - indexMitad + 3 + 4 + 1 = 8$

Por tanto obtenemos que $t_{it}(8) \in \Theta(1)$ y como es constante su coste se ve eclipsado por la combinación de los índices de posición, de los cuales solo van a haber como mucho n-4 índices para juntar.

En consecuencia obtenemos:

$$t_{ori}(n) = \begin{cases} C_1 \in \Theta(1) & n < 5 \quad \text{(no hay subcadenas posibles)} \\ t_{it}(n) \in \Theta(n) & 5 \leq n \leq 8 \quad \text{(se resuelve iterativamente)} \\ t_{ori}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + t_{ori}\left(\left\lceil \frac{n}{2} \right\rceil\right) + m(n-4) & n > 8 \quad \text{(divide y vencerás)} \end{cases}$$

Si asumimos:

$$\{n = 2^k \mid k > 0, k \in \mathbb{Z}^+\}$$

Se tiene que el algoritmo divide y vencerás es:

$$t(n) = \begin{cases} C_1 \in \Theta(1) & n < 5 \quad \text{(no hay subcadenas posibles)} \\ t_{it}(n) \in \Theta(n) & 5 \leq n \leq 8 \quad \text{(se resuelve iterativamente)} \\ 2t\left(\frac{n}{2}\right) + m(n-4) & n > 8 \quad \text{(divide y vencerás)} \end{cases}$$

5.4. Conclusión

Para obtener el orden del algoritmo divide y vencerás trabajaremos con:

$$t(n) = 2t\left(\frac{n}{2}\right) + m(n-4)$$

El **mejor caso** ocurre cuando ninguno de los caracteres de la cadena pertenece al conjunto S. En este escenario, el coste de `merge` $m(n-4)$ se mantiene constante, es decir, $m(n-4) \in \Theta(1)$, quedando informalmente como:

$$t(n) = 2\,t\left(\frac{n}{2}\right) + \Theta(1)$$

Como tenemos una ecuación de recurrencia de la forma $at(n/b) + f(n)$ podemos aplicar el Teorema Maestro.

- $a = b = 2$ por tanto $\log_2 2 = 1$
- $m(n-4) \in \Theta(1)$ donde $c = 0$
- Estamos ante el caso 1 donde $\log_b a > c$ porque $log_2 2 > 0 \equiv 1 > 0$

se obtiene que

$$t_m(n) \in \Theta(n).$$

El **peor caso** ocurre cuando la cadena tiene n-4 subcadenas que cumplen la condición del ejercicio. En este caso, el coste de `merge` $m(n-4)$ es lineal, es decir, $m(n-4) \in \Theta(n)$, quedando informalmente como:

$$t(n) = 2\,t\left(\frac{n}{2}\right) + \Theta(n)$$

Como tenemos una ecuación de recurrencia de la forma $at(n/b) + f(n)$ podemos aplicar nuevamente el Teorema Maestro y deducir que a partir de $t(n) = 2t(n/2) + \Theta(n)$ que:

- $a = b = 2$ por tanto $\log_2 2 = 1$
- $m(n-4) \in \Theta(n) \equiv \Theta(n^c)$ donde $c = 1$
- Estamos ante el caso 2 donde $\log_b a = c$

Se obtiene que

$$t_M(n) \in \Theta(n \cdot \log n)$$

Hasta ahora hemos estado suponiendo que n es potencia de 2. Sin embargo, podemos generalizarlo para cualquier n usando el *teorema de la b-armónica*:

- $t_{ori}(n)$ es eventualmente no decreciente.
- n y $n \cdot logn$ son crecientes.
- n y $n \cdot logn$ son 2-armónica.
- $t(n)$ es eventualmente no decreciente.
- $n \in \mathbb{Z}^+$.

Por tanto queda como: $t_{ori}(n) \in \Theta(t)$

6. Análisis experimental

Los tamaños de entrada considerados son $n_i = c_0 \cdot 2^i$ con $i = 0, \cdots, 10$ y $c_0 = 1000$. Se toma un salto multiplicativo para tener una mejor cobertura del domino, con estos saltos, cada punto es comparable al anterior ($n_{i+1}/n_i = 2$).
Si bien ya sabemos que el orden exacto teórico es n, lo cierto es que en la práctica si podemos establecer un mejor y un peor caso, el cual aumentará o reducirá la constante del merge.
El **mejor caso** es cuando no se encuentra ninguna cadena, porque así no se debe realizar ningún tipo de combinación de resultados.

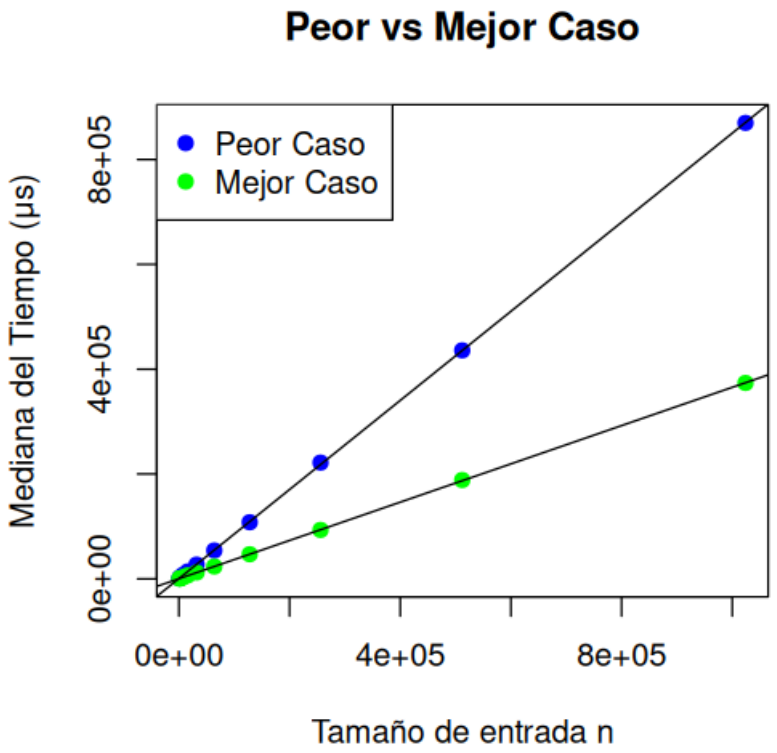
Algorithm 4 Generación de Mejor Caso

```
function GENERAR_MEJOR_CASO(tamEntrada, conjunto ← {a, b, c, d, e})
    cadena ← " "
    for i ← 0 to tamEntrada − 1 do
        cadena = cadena + 'z'
    end for
    return cadena
end function
```

El **peor caso** será lo contrario, cuando haya n-4 coincidencias y por tanto tengamos que estar combinando los resultados todo el rato.

Algorithm 5 Generación de Peor Caso

```
function GENERAR_PEOR_CASO(tamEntrada, conjunto ← {a, b, c, d, e})
    letraConjunto ← 0
    cadena ← " "
    for i ← 0 to tamEntrada − 1 do
        cadena = cadena + conjunto[letraConjunto]
        letraConjunto ← (letraConjunto + 1) mód conjunto.size()
    end for
    return cadena
end function
```



7. Contraste

Para contrastar los órdenes obtenido en el análisis teórico con las medidas obtenidas en el análisis experimental, se ha utilizado una simple regresión lineal.

- En el **mejor caso** obtenemos que $t_m(n) \in \Theta(n)$. Es decir, para n suficientemente grande, $t_m(n)$ se comportará de manera similar a $c \cdot n + d$, donde c y d son constantes. Llevado a la práctica, esto significa que podemos crear una regresión lineal usando los puntos (n_i, t_i) obtenidos en cada medida. A partir de estos puntos se puede calcular el coeficiente de determinación $R^2 \in [0, 1]$, que indica la calidad del ajuste.

- En el **peor caso** obtenemos que $t_M(n) \in \Theta(n \log n)$. Es decir, para n suficientemente grande, $t_M(n)$ se comportará de manera similar a $c \cdot n \log n + d$, donde c y d son constantes. En la práctica, esto implica realizar una regresión lineal utilizando los puntos $(n_i \log n_i, t_i)$ obtenidos en cada medición. A partir de ellos se puede calcular el coeficiente de determinación $R^2 \in [0, 1]$, que indica la calidad del ajuste.

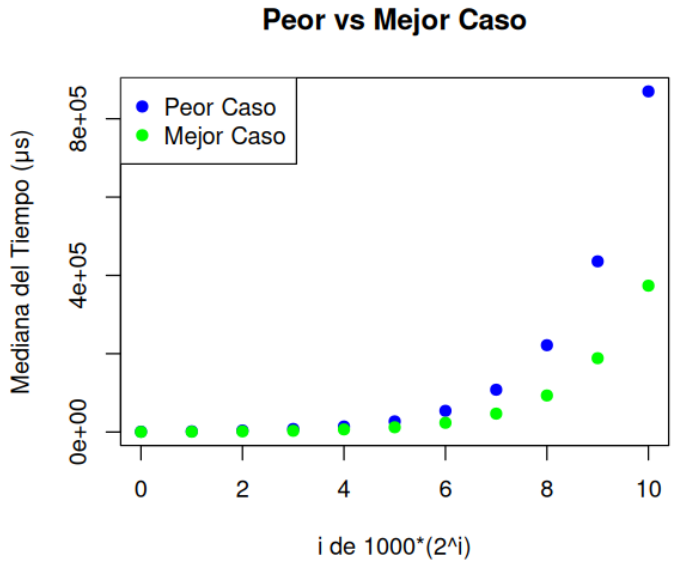


Figura 1: Medidas tomadas para el mejor y peor caso (Escala logarítmica)

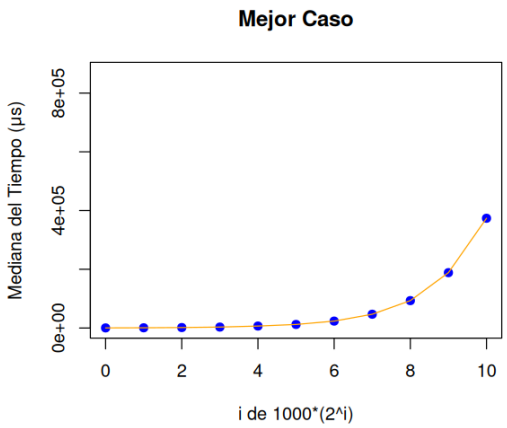


Figura 2: Ajuste al mejor caso

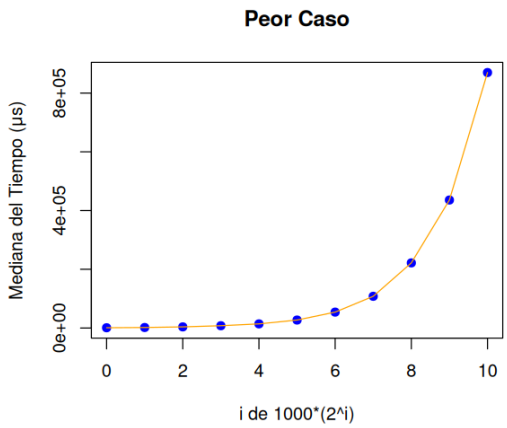


Figura 3: Ajuste al peor caso